

SPPL-262: Fluctuation Allowance Post API (Apply To All Groups False & Months True Only) technical workflow

Jira Ticket: [SPPL-262: BE | Analysis | Update technical workflow for Monthly FA Post \(Overall functionality\) API including DB details](#) **DONE**

- i** This page provides comprehensive information about the Post API for Monthly FA on the Input params page, to view Monthly FA details for the scenario by groups for all months.

The screenshot shows the 'cube' Global Planning System interface. The top navigation bar includes 'Plan & Forecast', 'Ordering', 'Demand & Supply Matching', 'Supply Planning', and 'Scheduling'. The breadcrumb trail is 'GPS Home / Vanning Rundown Summary / AP TMMK Line3 Aug2025 V1'. The main title is 'AP/TMMK Line 3/Aug 2025 V1'. A 'Mark Inputs as Ready' button is in the top right. The left sidebar has a 'Views' section with a search bar and a list of view types: 'Line Level Inputs', 'Vanning Center Inputs', 'Grouping Settings', 'Grouping', 'Min Max DoH', 'Fluctuation Allowance' (selected), 'Simulation', 'Review', and 'Reports'. The main content area is titled 'Fluctuation Allowance' and has a status of 'WORK IN PROGRESS'. It includes a 'See Master Data' button. The 'Monthly F/A' section has a toggle 'Apply to all months' and a value of 32%. The 'Daily F/A' section has a toggle 'Apply to all days' and a value of 20%. Below these are sections for 'Group 1 - TMH' and 'Group 2 - TMK'. At the bottom right are 'Cancel', 'Save', and 'Ready' buttons.

Supply Planning POST Monthly Fluctuation Allowance API Details:

1. **Hosted URL:** <https://api.spln.cube<env>.toyota.com/<env>>
2. **API Name:** <HOSTED_URL>/v/1/1/sp/monthly-fluctuation-allowance
3. **Path Param:** N/A
4. **Query Param:** N/A
5. **Method:** POST
6. **Authorization:** Bearer token (**required**)
7. **Lambda Name:** spln-<env>-update-monthly-fluctuation-allowance-v1-1
8. **API Request Body:**

```
1 {
2   "scenarioId": "uniqueScenarioId",
3   "userEmail": "user@toyota.com",
4   "applyTo": "BY_GROUPS",
5   "mode": "ALL_MONTHS",
6   "data": [
7     {
8       "groupId": "group1uuid",
9       "fa": { "value": 10 }
10    },
11    {
12      "groupId": "group2uuid",
```

```

13     "fa": { "value": 20 }
14   }
15 ]
16 }

```

9. API Response:

- **Success response with status 200:**

```

1 {
2   "message": "Successfully updated data."
3 }

```

- **Error Response:**

- **any internal server error occurred with status 500:**

```

1 {
2   "errorMessage": "Internal Server Error"
3 }

```

- **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```

1 {
2   "message": "Unauthorized"
3 }

```

- **If request is in-valid with status 400:**

```

1 {
2   "errorMessage": [<"ValidationError: validation error message">]
3 }

```

- **List of possible validation error message:**

1. "ValidationError: Request body cannot be empty."
2. "ValidationError: scenarioId is required and must be a uuid."
3. "ValidationError: userEmail is required and must be a string."
4. "ValidationError: Invalid userEmail."
5. "ValidationError: Scenario doesn't exist."
6. "ValidationError: applyTo is required and must be either ALL_GROUPS or BY_GROUPS."
7. "ValidationError: mode is required and must be either ALL_MONTHS or BY_MONTHS."
8. "ValidationError: data is required and must be an array with atleast 1 and atmost 15 of objects."
9. "ValidationError: groupId is required and must be a uuid."
10. "ValidationError: fa is required and must be an object."
11. "ValidationError: value is required and must be a number between 0 to 100."

10. **Table:**

- a. **scenarios**
- b. **grouping**
- c. **group_scenario_mapper**
- d. **monthly_fa**
- e. **scenario_step_status**

 SP Update Monthly Fluctuation Allowance Data API Workflow:

• **Technical workflow:**

- Description:
 - i. The Supply Planning user logs in to the application.
 - ii. After the user provides creates a scenario, opens it and navigates to Fluctuation Allowance, a request is sent to the SP Post Monthly Fluctuation Allowance endpoint: **sp/monthly-fluctuation-allowance** with below payload:

```
1 {  
2   "scenarioId": "uniqueScenarioId",  
3   "userEmail": "user@toyota.com",  
4   "applyTo": "BY_GROUPS",  
5   "mode": "ALL_MONTHS",  
6   "data": [  
7     {  
8       "groupId": "group1uuid",  
9       "fa": { "value": 10 }  
10    },  
11    {  
12      "groupId": "group2uuid",  
13      "fa": { "value": 20 }  
14    }  
15  ]  
16 }
```

- iii. After successful authentication, the **spln-update-monthly-fluctuation-allowance-v1-1** lambda is triggered by the SP update scenario Monthly Fluctuation Allowance endpoint: **sp/monthly-fluctuation-allowance**. If authentication fails, an error with status 401 (*refer: error response-**any unauthorized error occurred with status 401***) will be returned and sent to client.
- iv. After user authentication has been confirmed, the **spln-update-monthly-fluctuation-allowance-v1-1** lambda will perform validation on the request payload. If payload is valid will proceed to next step else throw validation error (*refer: error response: **Section if request is in-valid with status 400 and List of possible validation error message***). Below query is used to check if provided scenario is valid:

```
1 select * from supply_planning.scenarios where scenario_id = :scenarioId and
  is_active = true;
```

v. After the request payload is successfully validated, the **spln-update-fluctuation-allowance-v1-1** lambda execute below logic:

1. As the input is for “ALL_MONTHS” & “BY_GROUPS”, get active groupIds for scenario (from mapper) using below query:

```
1 SELECT sgm.group_id AS "groupId"
2 FROM supply_planning.group_scenario_mapper sgm
3 JOIN supply_planning.grouping gp
4 ON gp.group_id = sgm.group_id
5 WHERE sgm.scenario_id = ${scenarioId}::uuid
6 AND sgm.is_active = TRUE
7 AND gp.is_active = TRUE
8 ORDER BY sgm.group_id
```

2. Bulk UPDATE query with apply_to_all_months true — update monthly_fa table data by scenarioId, groupId using below query:

```
1 UPDATE TABLE supply_planning.monthly_fa SET
2     monthly_fa_percent = ${value}::int,
3     apply_to_all_months = false
4     updated_by = ${userEmail}::text,
5     last_updated_timestamp = CURRENT_TIMESTAMP
6 WHERE scenario_id = :scenarioId
7     AND group_id = :groupId;
```

3. Update the step & scenario status to “In Progress”. Only update scenario status to “In Progress” if its not already using scenarioData from query (iv) response.

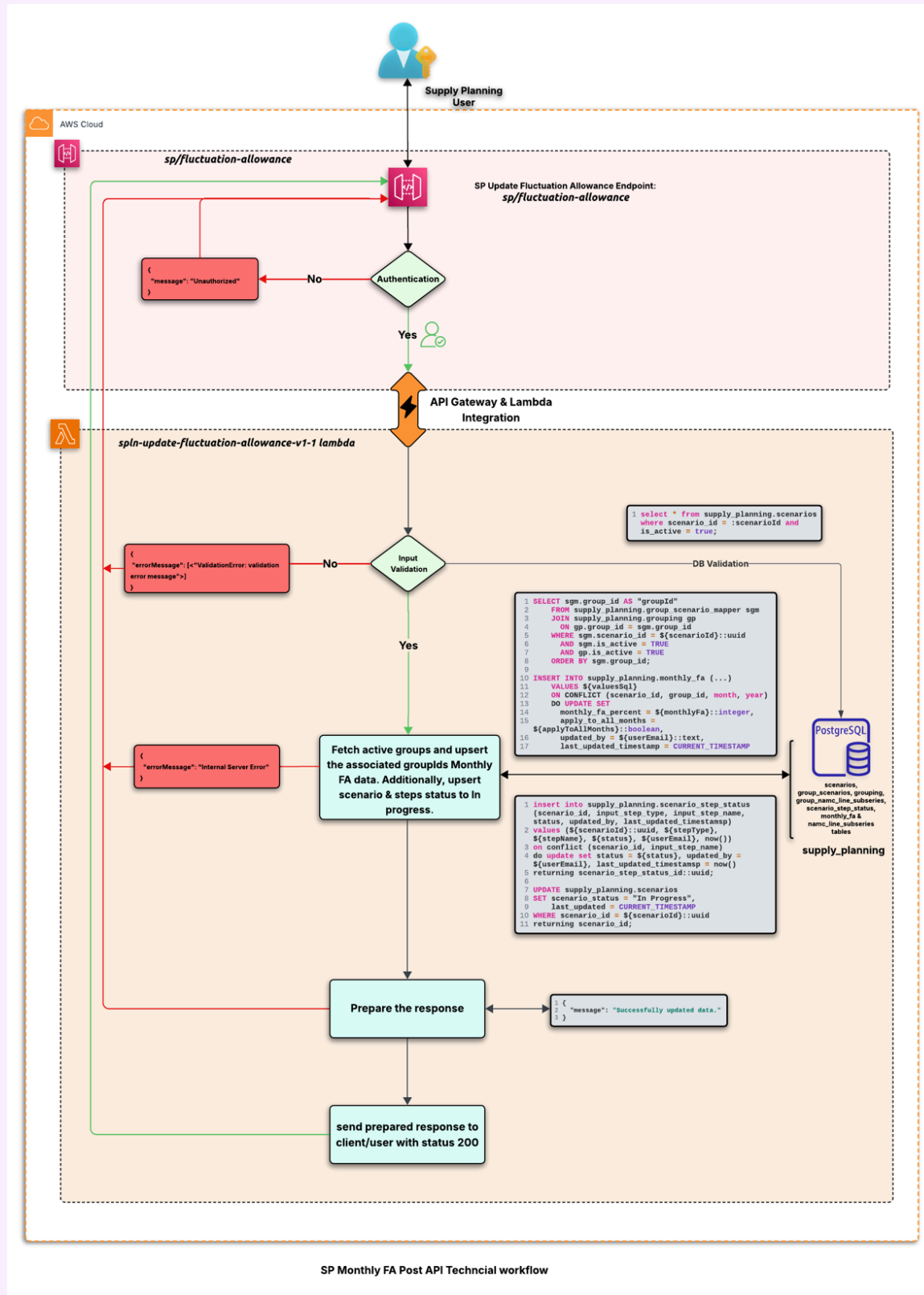
```
1 insert into supply_planning.scenario_step_status (scenario_id, input_step_type,
2 input_step_name, status, updated_by, last_updated_timestamp)
3 values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status}, ${userEmail},
4 now())
5 on conflict (scenario_id, input_step_name)
6 do update set status = ${status}, updated_by = ${userEmail},
7 last_updated_timestamp = now()
8 returning scenario_step_status_id::uuid;
9
10 UPDATE supply_planning.scenarios
11 SET scenario_status = "In Progress",
12     last_updated = CURRENT_TIMESTAMP
13 WHERE scenario_id = ${scenarioId}::uuid
14 returning scenario_id;
```

vi. After updating the Monthly FA & status data, return the below response to the client with a status code of 200:

```
1 {
2   "message": "Successfully updated data."
3 }
```

vii. If there is an error while fetching the data, an “Internal Server Error” with a status code of 500(refer: *error response-any internal server error occurred with status*

500) will be returned. Otherwise, a successful response with a status code of 200 will be sent.



Note:

We will be utilizing this API to update both file data and previous rundown data.

DB constraints:

```
CREATE INDEX IF NOT EXISTS idx_group_scenario_mapper_scenario_active  
ON supply_planning.group_scenario_mapper (scenario_id, is_active, group_id);
```

```
CREATE INDEX IF NOT EXISTS idx_monthly_fa_scenario_group_fa_month  
ON supply_planning.monthly_fa(scenario_id, group_id, fa_month);
```

Queries: